

자료구조

2분반

HW5

보고서

담당교수님 : 장형수 교수님

학과 : 컴퓨터공학과

학번: 20211562

이름: 이동협

문제1

문제 1은 linked binary tree로 max heap을 구현하여 input1.txt을 통해 주어진 insert, delete 명령을 수행하여 결과를 output1.txt에 출력하는 문제이다.

node *treePointer;
node *qpointer;
FILE *ifp, *ofp;
int exist

```

node{
    int key;
    treePointer parent;
    treePointer leftChild;
    treePointer rightChild;
}

```

```

queue{
    qpointer next;
    treePointer data;
}

```

```

void freeq(qpointer* q){
    qpointer curq = *q;
    while(curq != NULL){
        *q = curq->next;
        free(curq);
        curq = *q;
    }
}

```

```

treePointer new_node(){
    treePointer newnode = (treePointer)malloc(sizeof(node));
    newnode->key = key;
    newnode->leftChild = NULL;
    newnode->rightChild = NULL;
    newnode->parent = parent;
    return newnode;
}

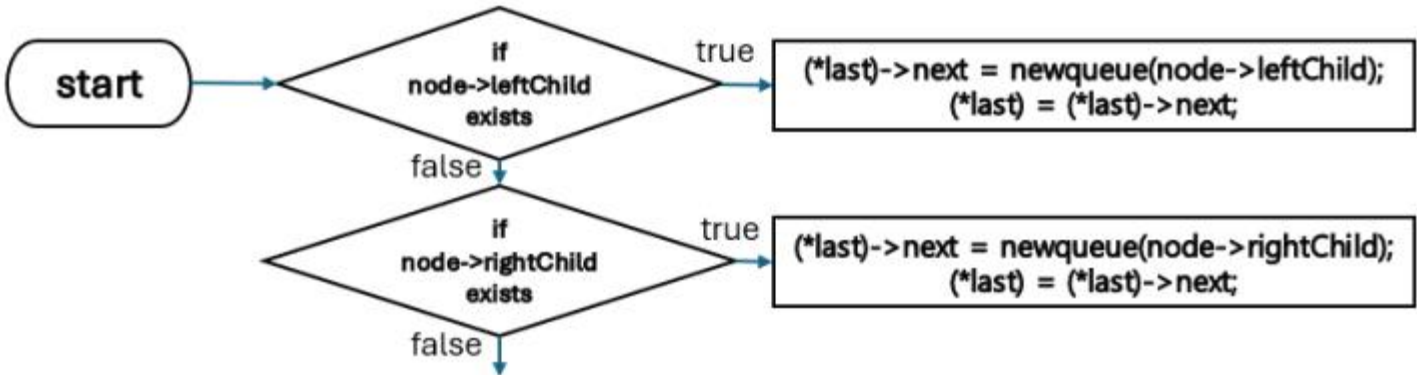
```

```

qpointer new_queue(){
    qpointer q = (qpointer)malloc(sizeof(queue));
    q->next = NULL;
    q->data = data;
    return q;
}

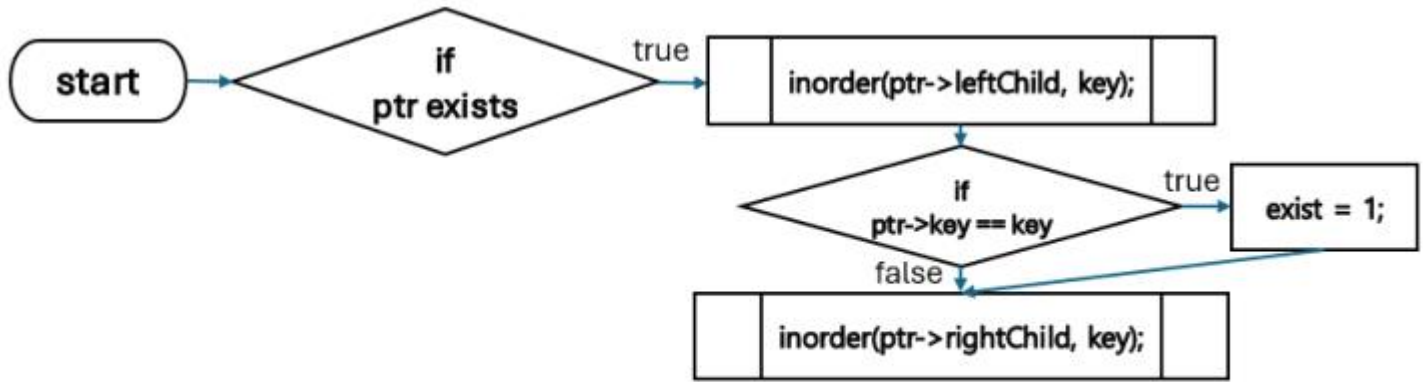
```

위의 code에서 동작하기 위해 필요한 data type과 treePointer를 initialize하는 new_node() 함수, qpointer를 initialize하는 new_queue()함수, qpointer를 메모리 해제하는 freeq이다. node는 해당 노드의 key값, 해당 노드의 parent, leftChild, rightChild treePointer를 멤버변수로 가진다. queue는 다음 queue값인 qpointer next, queue안에 들어있는 정보인 treePointer data를 멤버변수로 가진다. 또한 FILE *ifp, *ofp는 각각 입력파일, 출력파일의 파일포인터이며 int exist는 중복 원소를 체크하기 위한 flag이다.



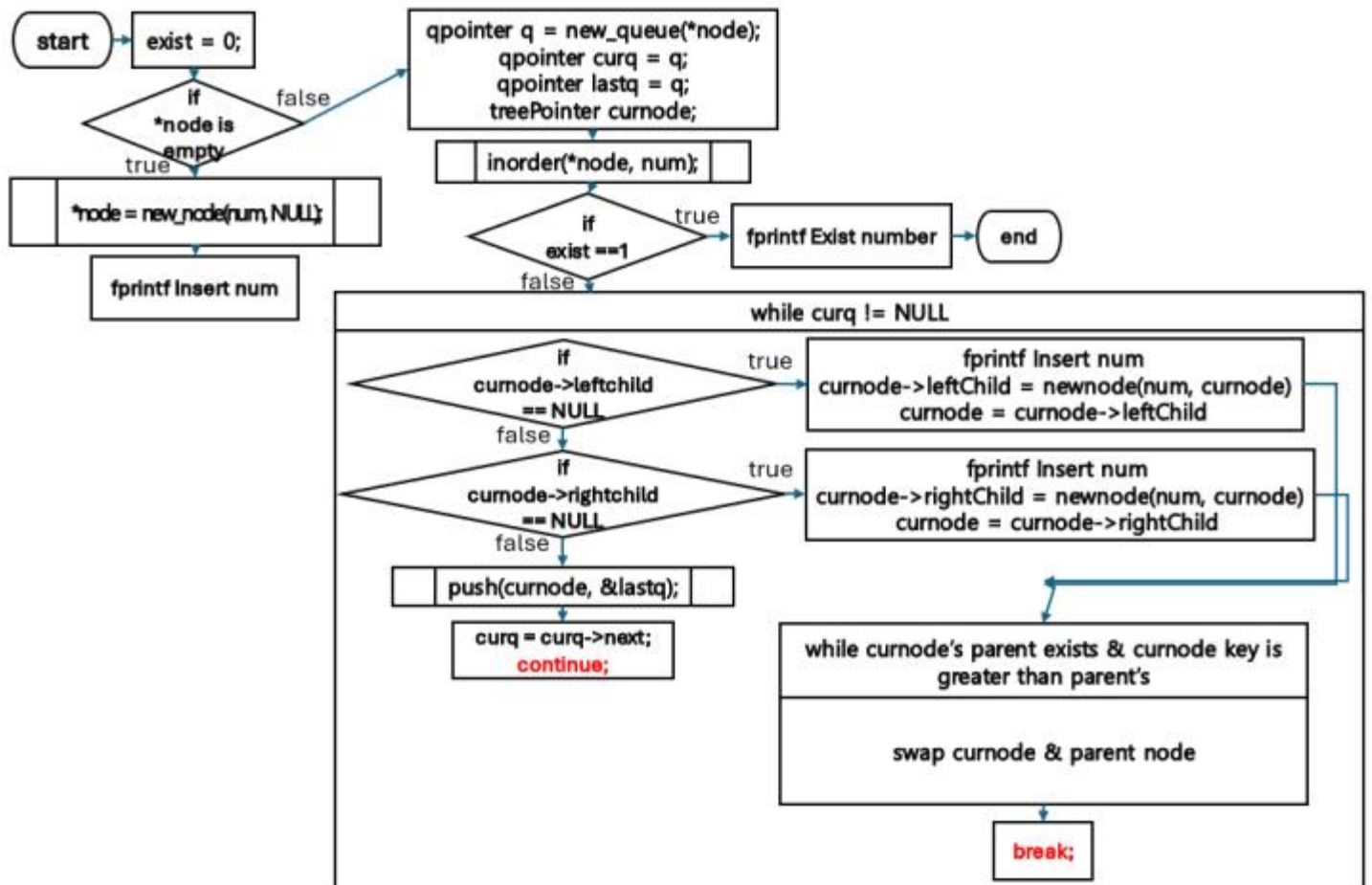
```
void push(treePointer node, qpointer* last)
```

push함수는 queue에 treePointer 값을 push하는 함수이다. node의 leftChild 값이 존재하면 queue의 다음 queue값에 leftChild값을 넣고, leftChild값이 존재하지 않고 rightChild값이 존재한다면 queue의 다음 queue값에 rightChild 값을 넣는다.



```
void inorder (treePointer ptr, int key)
```

inorder 함수는 inorder traversing으로 tree search를 하면서 key와 일치하는 값이 존재하는지 확인하는 함수이다. leftChild 값에 대해 inorder를 호출해주고 이후 같은 key값을 찾으면 exist flag를 1로 바꾼다. 같은 key값을 찾지 못하면 rightChild값에 대해 inorder를 더 호출해준다.



```
void insert(treePointer *node, int num)
```

insert 함수는 tree에 num을 key값으로 가지는 노드를 insert하는 함수이다.

먼저 exist flag를 0으로 초기화해주고 *node가 empty 인지 확인한다. empty라면 *node를 빈 노드로 initialize해주고 fprintf로 num이 insert 되었다고 출력해준다.

*node가 empty가 아니라면 동작에 필요한 queue들인 q(main queue), curq(current queue), lastq(last queue), treePointer curnode(currentnode)를 initialize해준다.

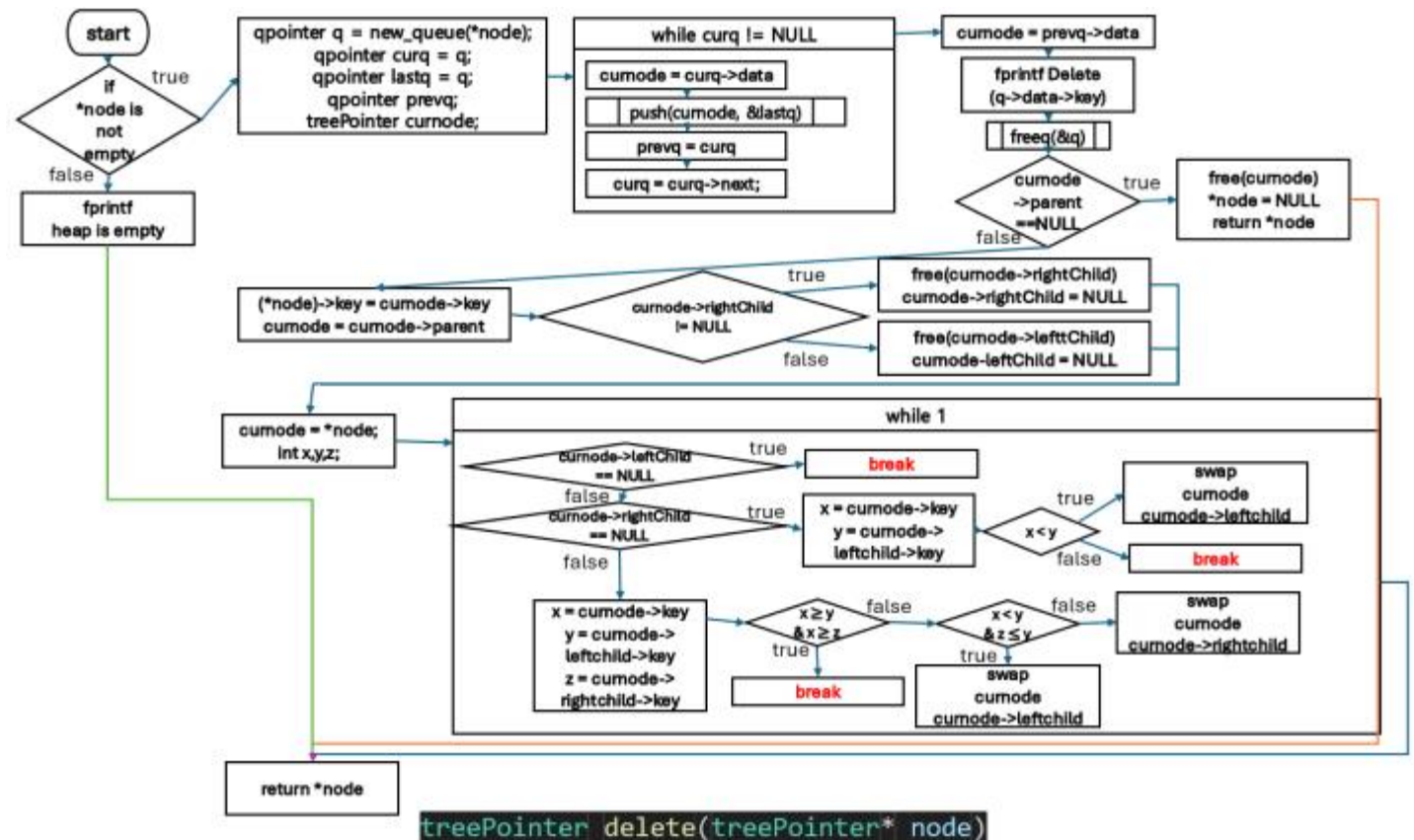
이후 num이 tree안에 존재하는지 확인하기 위해 inorder를 호출하여 중복 key가 존재했다면 fprintf로 중복 메시지를 출력하고 함수를 끝낸다.

중복 key가 존재하지 않았다면 while(curq != NULL) loop 안에서 insert 작업을 수행한다.

만약 현재 노드의 leftChild값이 없다면 현재 노드의 leftChild값에 num을 key로 가지는 treeNode를 insert해주고 현재 노드의 위치를 leftChild로 옮긴다.(동시에 fprintf로 Insert num도 출력해준다.) 이후 현재 노드가 최상위 노드가 아니고 현재 노드의 key값이 parent 노드의 값보다 크다면 계속 parent 노드로 올라가면서 현재 노드와 parent 노드를 swap해준다. 이후 while(curq != NULL) loop에서 탈출한다.

만약 현재 노드의 rightChild값이 없다면 현재 노드의 rightChild값에 num을 key로 가지는 treeNode를 insert해주고 현재 노드의 위치를 rightChild로 옮긴다.(동시에 fprintf로 Insert num도 출력해준다.) 이후 현재 노드가 최상위 노드가 아니고 현재 노드의 key값이 parent 노드의 값보다 크다면 계속 parent 노드로 올라가면서 현재 노드와 parent 노드를 swap해준다.이후 while(curq != NULL) loop에서 탈출한다.

만약 현재 노드의 leftChild rightChild 값이 모두 있다면 현재 노드를 lastq에 push해준다. 이후 curq를 다음 queue 값으로 이동시키고 while loop의 다음 반복으로 이동한다.



delete 함수는 tree에서 가장 큰 key 값을 가지는 node를 delete(max heap에서 delete는 가장 큰 key 값을 제거하는 것이기때문에)하는 함수이다.

가장 먼저 argument로 들어온 node값을 확인하여 empty라면 fprintf로 heap is empty를 출력해준다.

node가 empty가 아니라 노드가 존재한다면 동작에 필요한 queue들인 q(main queue), curq(current queue), lastq(last queue), prevq(previous queue), treePointer curnode(currentnode)를 initialize해준다.

이후 while curq != NULL loop에서 curnode에 curq의 data를 저장하고 curnode를 lastq에 push하고 prevq에 curq를 저장하고 curq는 next로 이동하는 작업을 반복한다.

while loop이 끝나면 curnode에 prevq의 data를 저장한다. 그리고 fprintf로 q의 data에 있는 key값이 delete되었다고 출력한다.

이후 curnode의 parent가 없다면 curnode를 free하고 NULL node를 return한다.

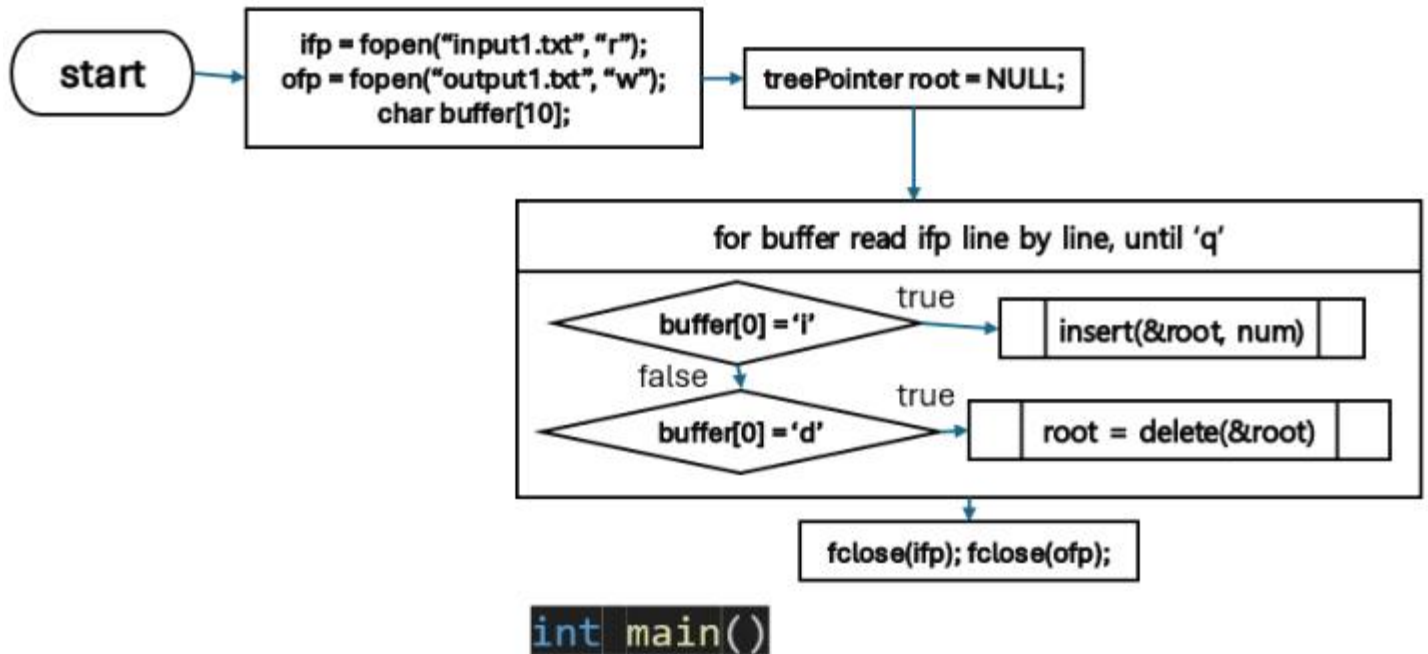
curnode의 parent값이 존재하는 경우 curnode의 key값을 node의 key값에 저장하고 curnode를 curnode의 parent 노드로 이동한다.

이후 만약 curnode의 rightChild값이 존재하는 경우 rightChild를 free하고, rightChild값이 없는 경우 leftChild를 free한다.

이후 curnode에 node를 저장하고 delete된 노드 때문에 엉킨 order를 재배치해준다.

아래 실행문들은 while 1에서 break를 만날 때까지 무한반복한다.

1. curnode의 leftChild가 없다면 break
2. curnode의 rightChild가 없다면 curnode의 key값과 leftChild의 key 값을 비교하여 leftChild값이 더 크다면 curnode와 leftChild 값을 swap한다. 만약 curnode의 값이 더 크다면 break
3. leftChild, rightChild가 모두 존재한다면 curnode, leftChild, rightChild key값을 비교하여 curnode값이 leftChild, rightChild 값보다 크다면 break
leftChild값이 curnode, rightChild 값보다 크다면 leftChild와 curnode를 swap해준다.
위 두 경우가 아니라면 rightChild와 curnode를 swap해준다.



main 함수에서는 fopen으로 입력, 출력파일인 input1.txt, output1.txt를 각각 열어주고 root node인 treePointer root를 NULL값으로 선언해준다. 이후 파일을 한줄씩 읽어가면서 i(insert)가 입력되면 같은 줄에 입력된 숫자를 num에 저장하여 insert(&root, num)과 같은 형식으로 root에 insert해준다. d(delete)가 입력되면 delete(&root)으로 호출하여 root의 최대값을 제거한다. 이후 input1.txt에서 q를 읽은 경우 읽기를 끝내고 ifp, ofp파일을 모두 닫고 끝낸다.

```

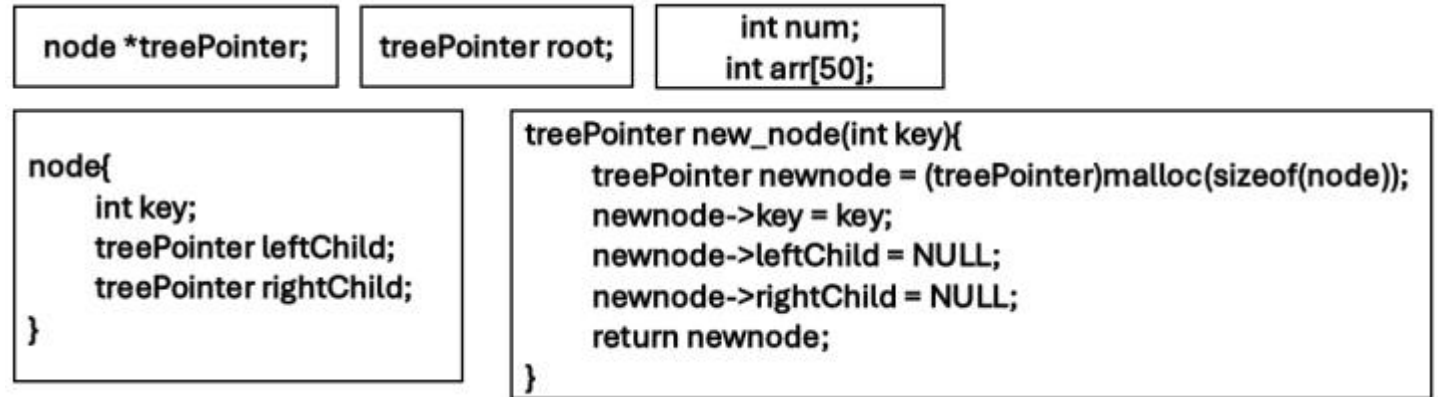
cse20211562@cspro1:~/structure/hw5$ cat input1.txt
i 4
i 4
i 5
d
d
d
i 3
qcse20211562@cspro1:~/structure/hw5$ gcc 1.c
cse20211562@cspro1:~/structure/hw5$ ./a.out
cse20211562@cspro1:~/structure/hw5$ cat output1.txt
Insert 4
Exist number
Insert 5
Delete 5
Delete 4
The heap is empty
Insert 3

```

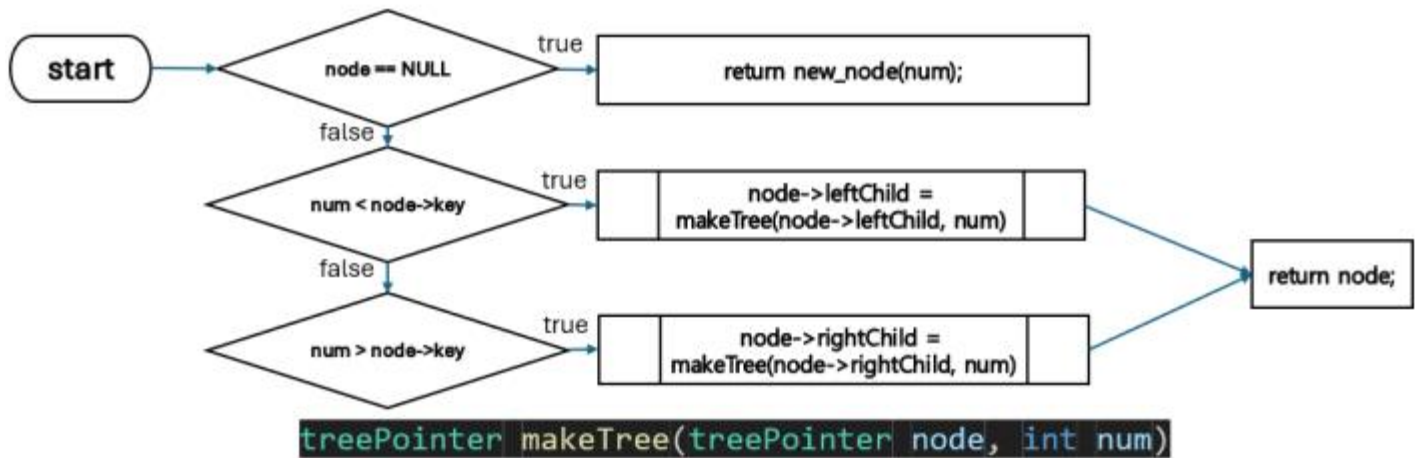
이는 제시된 출력 예시를 만족한다.

문제2

문제 2는 input2.txt에서 주어진 preorder traversal을 통해 BST를 만들고 이에 대해 inorder, postorder traversal을 수행하는 문제이다.



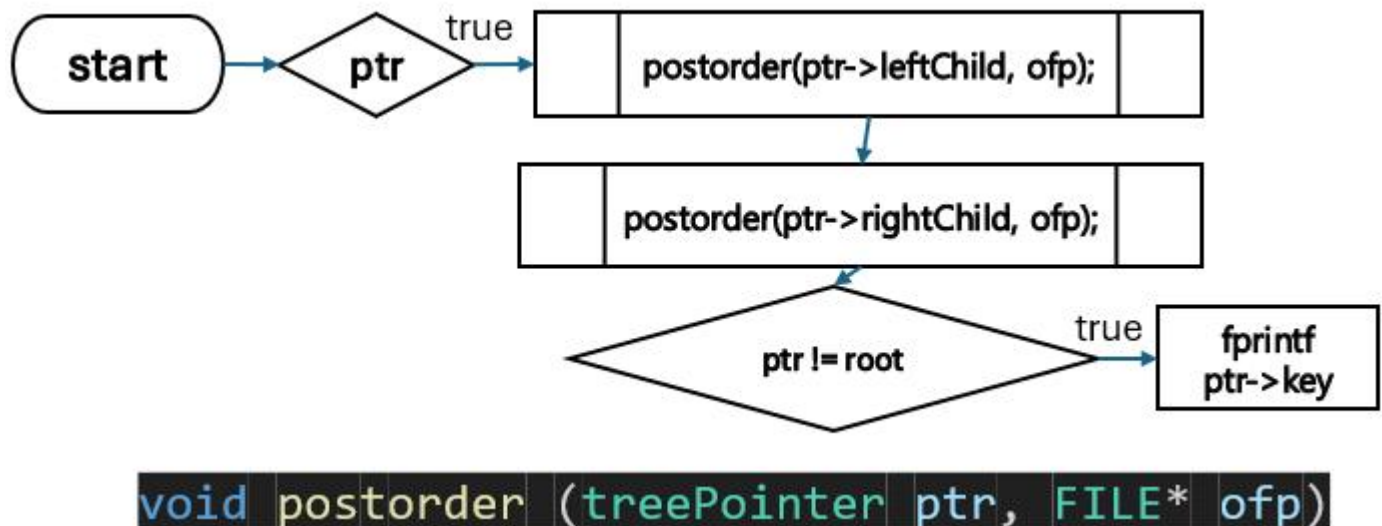
위의 code에서 동작하기 위해 필요한 data type과 treePointer를 initialize하는 new_node() 함수이다. node는 해당 노드의 key값, 해당 노드의 leftChild, rightChild treePointer를 멤버변수로 가진다. num은 input2.txt에서 입력받은 숫자의 총 개수를 저장하는 변수이며 arr[50]은 입력받은 각 숫자를 저장하는 변수이다.



makeTree 함수는 argument로 받은 num을 Tree에 key값으로 insert하는 함수이다.

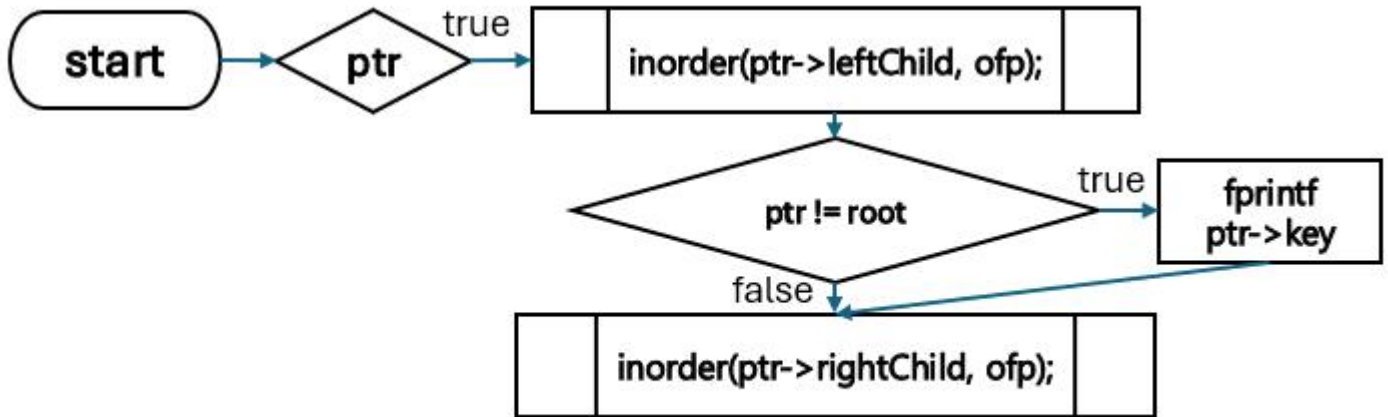
treePointer node가 비어있다면 new_node(num)을 통해 새로운 노드에 num을 insert하여 return 해준다.

node가 비어있지 않고 node의 key값이 num보다 크다면 node->leftChild를 이용하여 makeTree를 재귀적으로 호출해준다. node의 key값이 num보다 작다면 node->rightChild를 이용하여 makeTree를 재귀적으로 호출해준다.



postorder 함수는 postorder로 tree를 traversing하는 결과를 출력하는 함수이다.

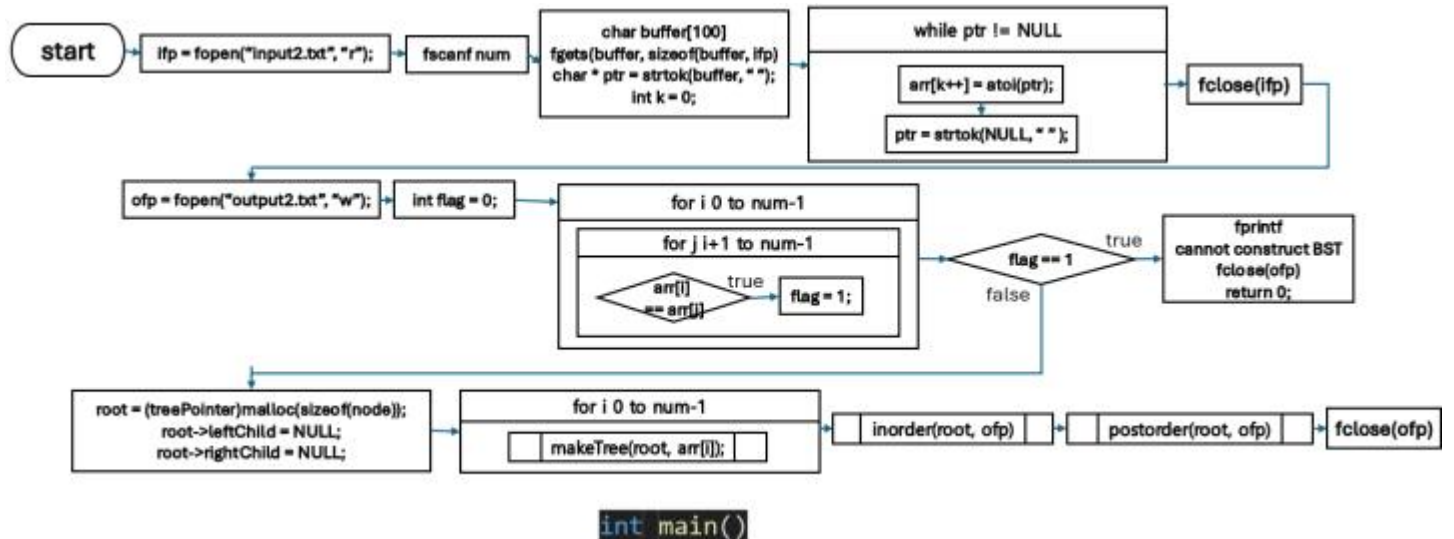
postorder의 argument로 들어온 ptr이 값이 존재한다면 ptr->rightChild를 이용하여 postorder를 재귀적으로 호출하고 이후 ptr->rightChild를 재귀적으로 호출한다. 이후 ptr이 root와 다른 노드라면 ptr의 key값을 fprintf 해준다.



```
void inorder (treePointer ptr, FILE* ofp)
```

inorder 함수는 inorder로 tree를 traversing하는 결과를 출력하는 함수이다.

inorder의 argument로 들어온 ptr이 값이 존재한다면 ptr->leftChild를 이용하여 inorder를 재귀적으로 호출한다. 이때 ptr이 root와 다른 노드라면 ptr의 key값을 fprintf 해준다. 이후 ptr->rightChild를 재귀적으로 호출한다.



main함수는 input2.txt에서 tree의 element를 읽고 inorder, postorder traverse한 결과를 output3.txt에 쓴다.

먼저 input2.txt를 열고 숫자의 개수를 num에 저장하고 다음줄을 읽는다. 다음줄의 숫자들을 " "단위로 잘라 arr에 하나씩 넣는다. 이후 input2.txt를 닫는다.

이후 output2.txt를 열고 중복 원소가 있는지 확인하는 int flag를 0으로 초기화한다.

arr에서 중복되는 수가 있는지 확인하여 중복이 있다면 fprintf로 cannot construct BST를 출력하고 파일을 닫고 끝낸다.

중복되는 수가 없다면 tree의 root가 될 treePointer root를 선언하고 makeTree를 이용하여 root에 arr에 있는 수들을 하나씩 insert한다.

이후 inorder, postorder를 이용하여 각 결과를 output2.txt에 쓰고 파일을 닫는다.

main 함수는 a.txt와 b.txt에서 polynomial 정보를 읽어 polypointer a, b에 각각의 polynomial을 나타내는 linkedlist를 저장한다.

가장 먼저 a.txt를 열어 첫줄에서 element의 개수를 읽어 저장하고 element의 개수만큼 한줄씩 element의 coefficient, exponential 값을 읽어 a에 연결해준다. 모두 끝나면 파일을 닫는다.

이후 b.txt를 열어 첫줄에서 element의 개수를 읽어 저장하고 element의 개수만큼 한줄씩 element의 coefficient, exponential 값을 읽어 b에 연결해준다. 모두 끝나면 파일을 닫는다.

```
cse20211562@cspro1:~/structure/hw5$ cat input2.txt
6
30 5 2 40 35 80cse20211562@cspro1:~/structure/hw5$ gcc 2.c
cse20211562@cspro1:~/structure/hw5$ ./a.out
cse20211562@cspro1:~/structure/hw5$ cat output2.txt
Inorder: 2 5 30 35 40 80
Postorder: 2 5 35 80 40 30
```

```
cse20211562@cspro1:~/structure/hw5$ cat input2.txt
6
30 5 5 40 35 80cse20211562@cspro1:~/structure/hw5$ gcc 2.c
cse20211562@cspro1:~/structure/hw5$ ./a.out
cse20211562@cspro1:~/structure/hw5$ cat output2.txt
cannot construct BST
```

제시된 출력 예시를 만족한다.

문제3

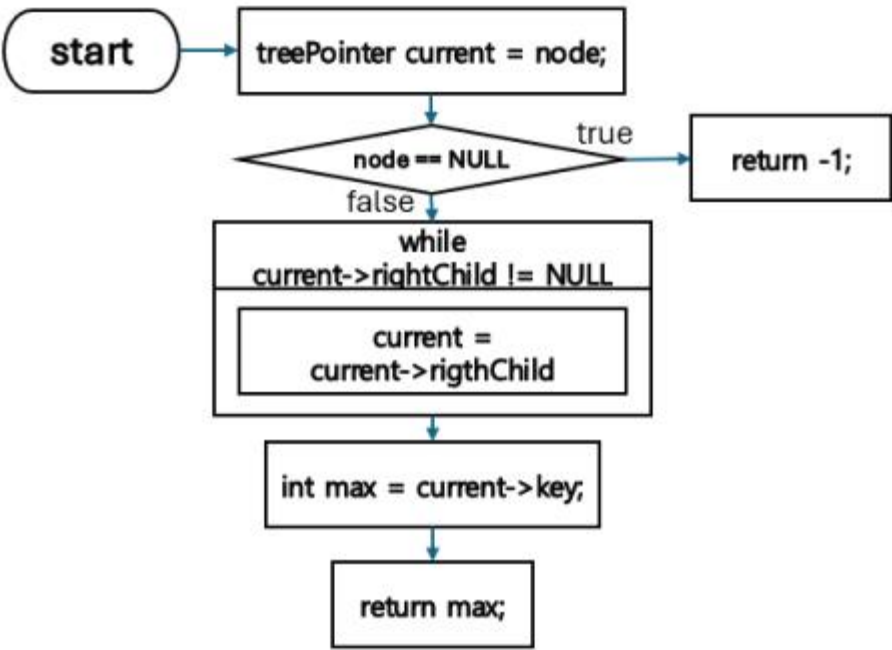
문제 3은 BST를 통해 max priority queue를 구현하고, input3.txt를 통해 주어진 push, top, pop을 수행한 결과를 output3.txt에 출력하는데 이때 top, pop, push의 시간복잡도는 $O(h)$ (h 는 search tree의 height)를 만족해야하는 문제이다.

```
node *treePointer;
```

```
node{
    int key;
    treePointer leftChild;
    treePointer rightChild;
}
```

```
treePointer new_node(int key){
    treePointer newnode = (treePointer)malloc(sizeof(node));
    newnode->key = key;
    newnode->leftChild = NULL;
    newnode->rightChild = NULL;
    return newnode;
}
```

문제3을 해결하기 위해 사용한 node, node를 가리키는 treePointer, treePointer를 initialize하는 new_node함수이다. node는 멤버변수로 key값, leftChild, rightChild treePointer를 가진다.



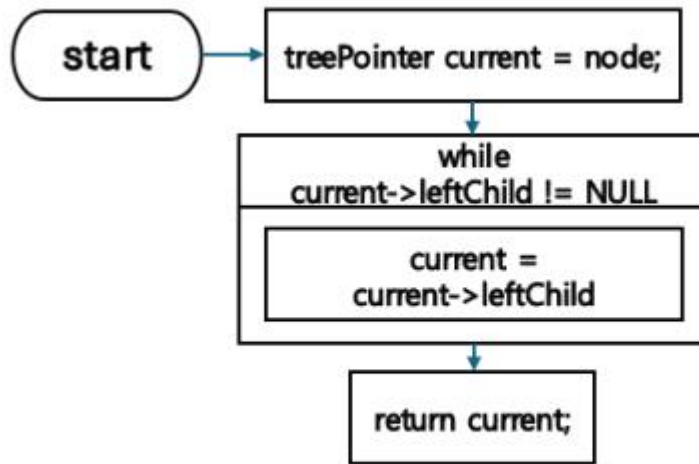
```
int max_node(treePointer node)
```

max_node 함수는 tree에서 가장 큰 key값을 return하는 함수이다. 표면적으로는 top함수와 같은 동작을 하나 key값만 다른 함수에서도 쓸 일이 있어 따로 만들었다.

treePointer current를 선언하여 node를 저장하고 node가 NULL이라면 -1(에러값)을 return한다.\

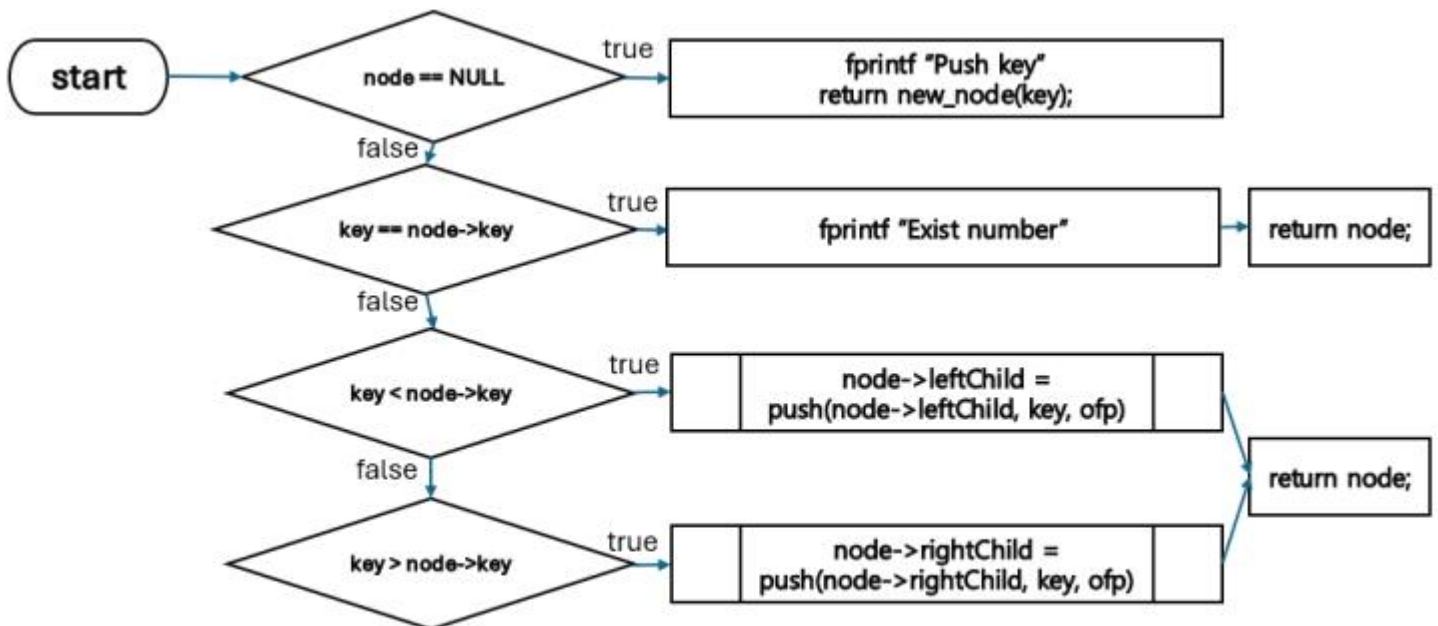
만약 node에 값이 있다면 current의 가장끝 rightChild 값으로 이동하여 그 key 값을 반환한다.

시간복잡도는 $O(h)$ 이다.(tree의 가장끝 rightChild값으로 이동할 때 h 만큼만 가므로)



```
treePointer min_node(treePointer node)
```

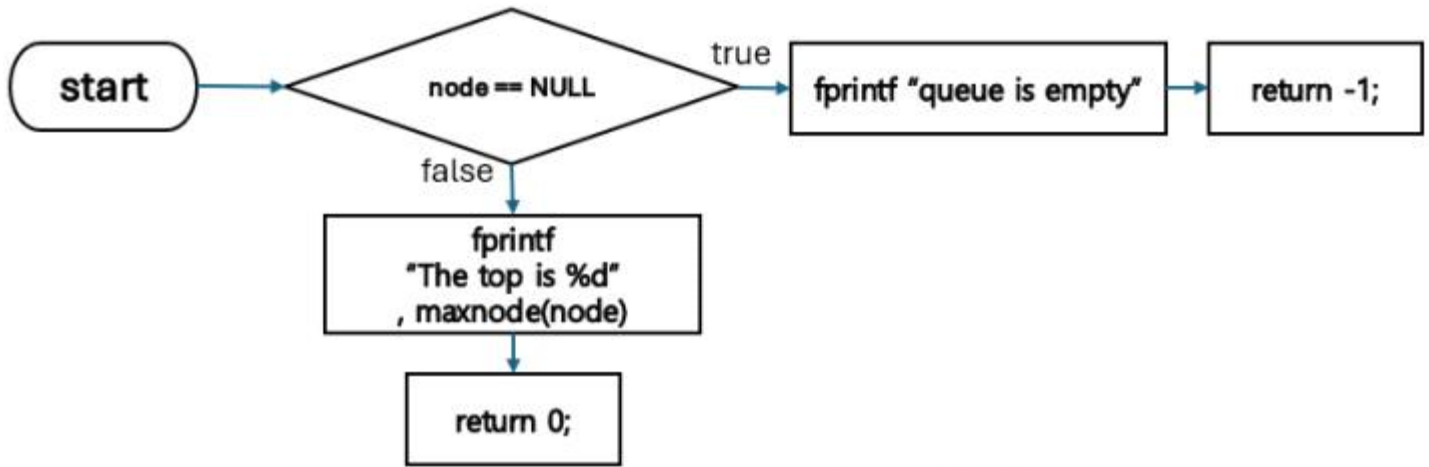
min_node함수는 tree에서 가장 작은 key값을 가진 node의 pointer를 return하는 함수이다.
 treePointer current에 node를 저장해주고 node의 가장 끝 leftChild로 이동하여 해당 노드를 return한다.
 시간복잡도는 $O(h)$ 이다.(tree의 가장 끝 leftChild값으로 이동할 때 h 만큼만 가므로)



```
treePointer push(treePointer node, int key, FILE* ofp)
```

push 함수는 tree에 key값을 push하는 함수이다.

- 1) node가 NULL이라면 printf로 push한 key값을 출력해주고 key값을 가지는 새로운 노드를 return한다.
 - 2) node의 key값이 key(argument)와 같다면 중복 key가 push 되는 것이므로 printf로 exist number를 출력하고 node를 그대로 return한다.
 - 3) node의 key값이 key보다 크다면 node->leftChild를 이용하여 push함수를 재귀적으로 호출해준다.
 - 3) node의 key값이 key보다 작다면 node->rightChild를 이용하여 push함수를 재귀적으로 호출해준다.
- push 함수는 leftChild나 rightChild로만 이동하며 node가 NULL이 됐을 때 멈추므로 $O(h)$ 의 시간복잡도를 가진다.



```
int top(treePointer node, FILE* ofp)
```

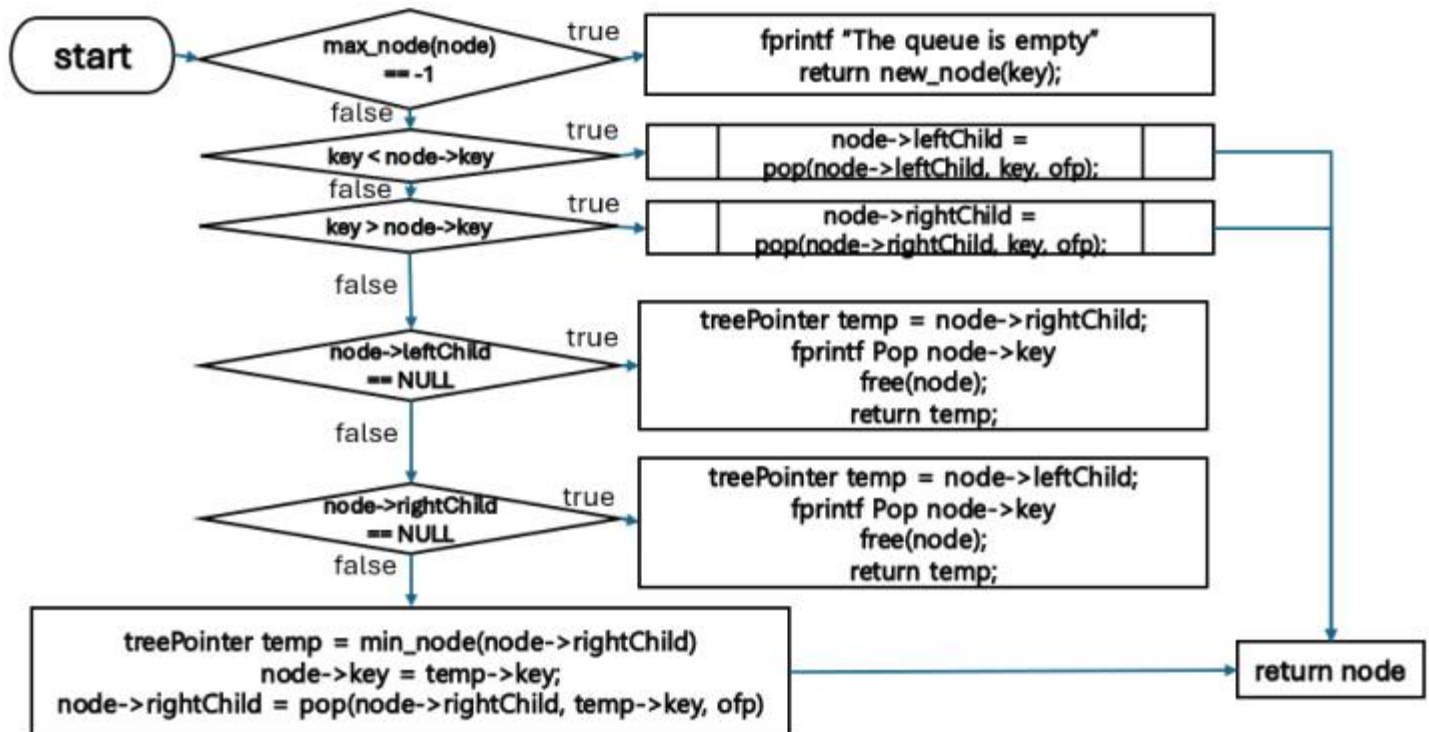
top 함수는 tree의 가장 상단 노드의 key 값을 출력하는 함수이다.

node가 NULL일 경우 fprintf로 queue is empty를 출력해주고 -1을 return한다.(오류값)

node가 존재할 경우 maxnode(node)를 통해 얻은 값을 fprintf로 출력해준다.

*maxnode로 얻은 값을 출력하는 이유는 max priority queue의 top 값은 가장 큰 값이기 때문이다.

*top의 시간복잡도는 반복문이 없으므로 maxnode의 시간복잡도를 따르는데 maxnode의 시간복잡도가 $O(h)$ 이므로 top의 시간복잡도 또한 $O(h)$ 이다.



```
treePointer pop(treePointer node, int key, FILE* ofp)
```

pop 함수는 tree에서 가장 큰 값을 가지는 node를 제거하면서 해당 node의 key 값을 출력하는 함수이다.

가장 먼저 node의 max_node값이 -1이라면(node가 NULL이라면) fprintf로 the queue is empty를 출력하고 key 값 (애초에 이 함수에 key값으로 maxnode값이 들어오므로 거짓값을 가진 node가 return된다.)을 가진 node를 return한다.

node가 비어있지 않고 node의 key값이 key(argument)보다 크다면 node->leftChild를 이용하여 pop을 재귀적으로 호출해준다.

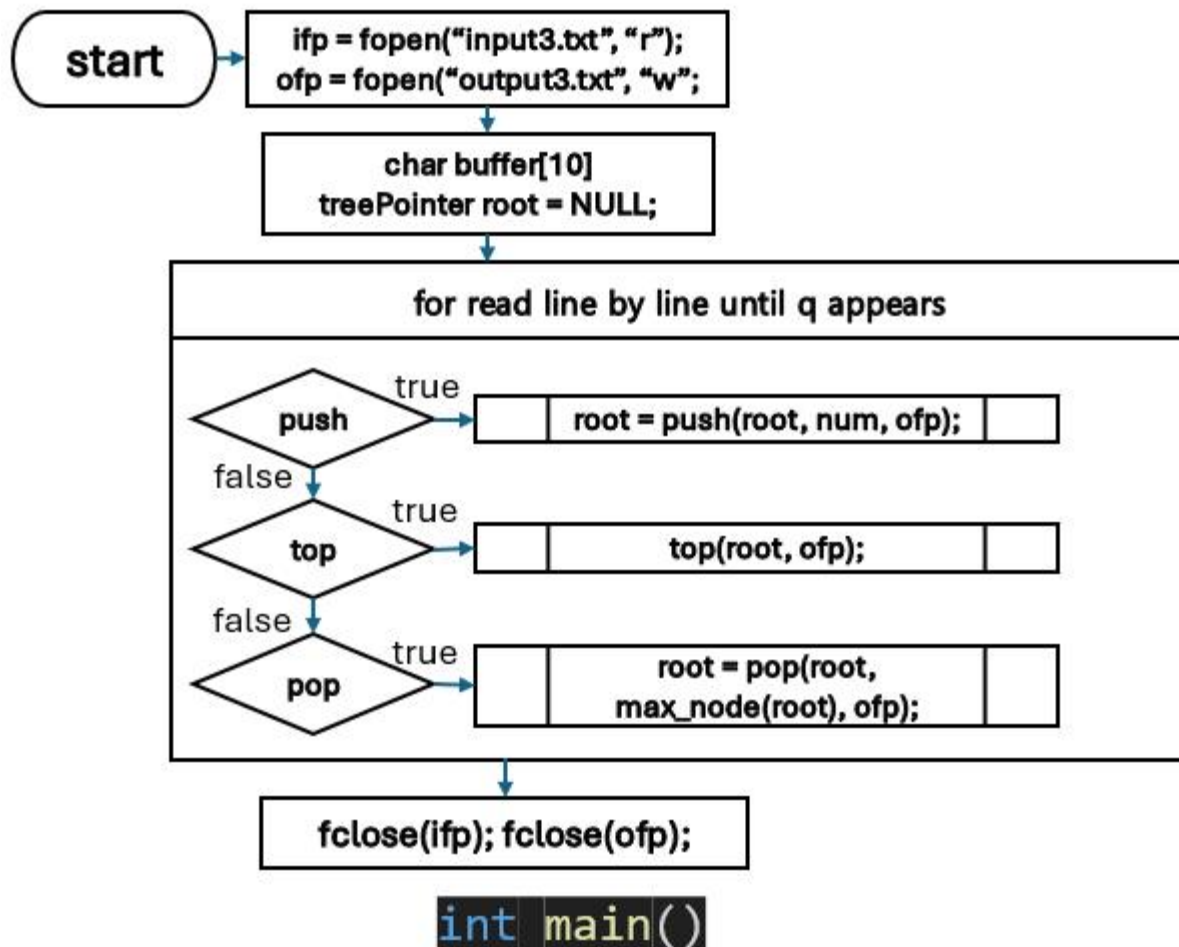
node가 비어있지 않고 node의 key값이 key(argument)보다 작다면 node->rightChild를 이용하여 pop을 재귀적으로 호출해준다.

위의 조건들이 모두 false이고 (앞선 pop의 재귀호출로 key값이 같다면) node의 leftChild가 비어있다면 treePointer temp에 rightChild값을 저장하고 fprintf로 node의 key값을 출력하고 node를 free한 후 temp를 return한다.

위의 조건들이 모두 false이고 (앞선 pop의 재귀호출로 key값이 같다면) node의 rightChild가 비어있다면 treePointer temp에 leftChild값을 저장하고 fprintf로 node의 key값을 출력하고 node를 free한 후 temp를 return한다.

위의 조건들이 모두 false이고 leftChild, rightChild값이 존재한다면 treePointer temp에 rightChild의 minnode값을 저장하고 그 minnode의 key값을 node에 저장한다. 이후 node->rightChild를 이용하여 pop을 재귀적으로 호출한다.

pop 함수는 반복문이 없고 시간복잡도가 $O(h)$ 인 max_node, min_node의 호출, pop의 재귀호출밖에 없다. pop함수는 재귀호출 동안 tree에서 height만큼만 움직이므로 $O(h)$ 를 만족한다.



main 함수는 input3.txt에서 한줄씩 각 명령을 읽어 이에 대한 결과를 output3.txt에 출력하는 함수이다.

fclose를 통해 input3.txt, output3.txt를 열고 tree의 root가 될 treePointer root를 NULL로 선언한다.

이후 q가 나올 때까지 파일을 한줄씩 읽으면서 push라면 `push(root, num, ofp)`값을 root에 저장해주고 top이라면 `top(root, ofp)`를 실행하고 pop이라면 `pop(root, max_node(root), ofp)`값을 root에 저장해준다. (*pop에 `max_node(root)`를 argument로 넣는 이유는 max priority queue의 특성상 가장 큰 값이 pop되기 때문이다.

```
cse20211562@cspro1:~/structure/hw5$ cat input3.txt
push 3
push 3
top
push 5
top
pop
push 3
pop
pop
cse20211562@cspro1:~/structure/hw5$ gcc 3.c
cse20211562@cspro1:~/structure/hw5$ ./a.out
cse20211562@cspro1:~/structure/hw5$ cat output3.txt
Push 3
Exist number
The top is 3
Push 5
The top is 5
Pop 5
Exist number
Pop 3
The queue is empty
```

이는 제시된 출력 예시를 만족한다.